

DSA Coach — Editor + KeyStroke Agent

Adaptive Coding-Behaviour Intelligence for the DSA Coach Platform

Team Prometheus | GENAICH-003

1. Project Overview

DSA Coach is an AI-powered personalized learning platform that helps students learn Data Structures and Algorithms through interactive, step-by-step guidance. Within this larger platform, the Editor Agent and KeyStroke Agent form the sensing layer that sits directly inside the student's coding environment — they are the component that actually watches how a student writes code, not just what code they eventually submit.

Team Prometheus owns this component: a paired Editor Agent (understands the code itself) and KeyStroke Agent (understands the behaviour behind the code) that together convert raw typing activity into a structured signal the rest of DSA Coach can act on.

Core Technologies

- FastAPI — backend API and WebSocket layer for live editor events
- Monaco-based Code Editor — captures per-keystroke and per-edit events in the browser
- FAISS — vector store for DSA knowledge retrieval (RAG completed)
- Redis — live keystroke buffer and active session state
- PostgreSQL — persistent typing analytics and submission history
- LLMs (Gemini Pro, Groq + Llama 3) — behaviour-aware feedback generation
- CrewAI-style Orchestrator — routes editor events to the correct agent

2. Problem Statement

Existing DSA learning tools evaluate a student only after code is submitted. This creates several gaps:

- No visibility into how a student arrived at a solution — only the final code is seen.
- Pasted or copied solutions look identical to genuinely authored code at submission time.
- Struggling students who freeze mid-problem receive no help until they give up or submit.
- Hint systems cannot tell the difference between a student who is thinking and a student who is stuck.
- There is no continuous behavioural signal to calibrate how much help is genuinely needed.

Solution

The Editor Agent and KeyStroke Agent continuously observe the live editing session — code structure changes and typing behaviour — and convert both into a single, real-time "editor context" that downstream agents (Hint, Evaluator, RAG) can use to respond intelligently, instead of waiting for a final submission.

3. Project Objectives

1. Capture code-editor state changes in real time without interrupting the student.
2. Capture keystroke-level typing behaviour: cadence, pauses, deletions, and paste events.
3. Distinguish authored typing from pasted content.
4. Detect hesitation and struggle patterns from typing rhythm alone.
5. Track structural code changes (diffs) independent of raw keystrokes.

6. Merge both signals into a single feature object for the Hint and Evaluator agents.
7. Retrieve relevant DSA context using RAG (FAISS) to ground any generated nudges.
8. Preserve student privacy — store behavioural signals, not raw keystroke content.

4. Project Scope

The system covers:

- Live code-editor event capture
- Keystroke dynamics capture and feature extraction
- Paste / bulk-insert detection
- Code structure diffing
- Struggle / hesitation scoring
- Feature aggregation for downstream agents
- RAG-based knowledge grounding (FAISS)
- Session and analytics storage

Out of scope for this component (owned by other GENAICH teams): final answer evaluation, hint content generation, code execution, and the proctoring pipeline.

5. System Architecture

High-Level Architecture

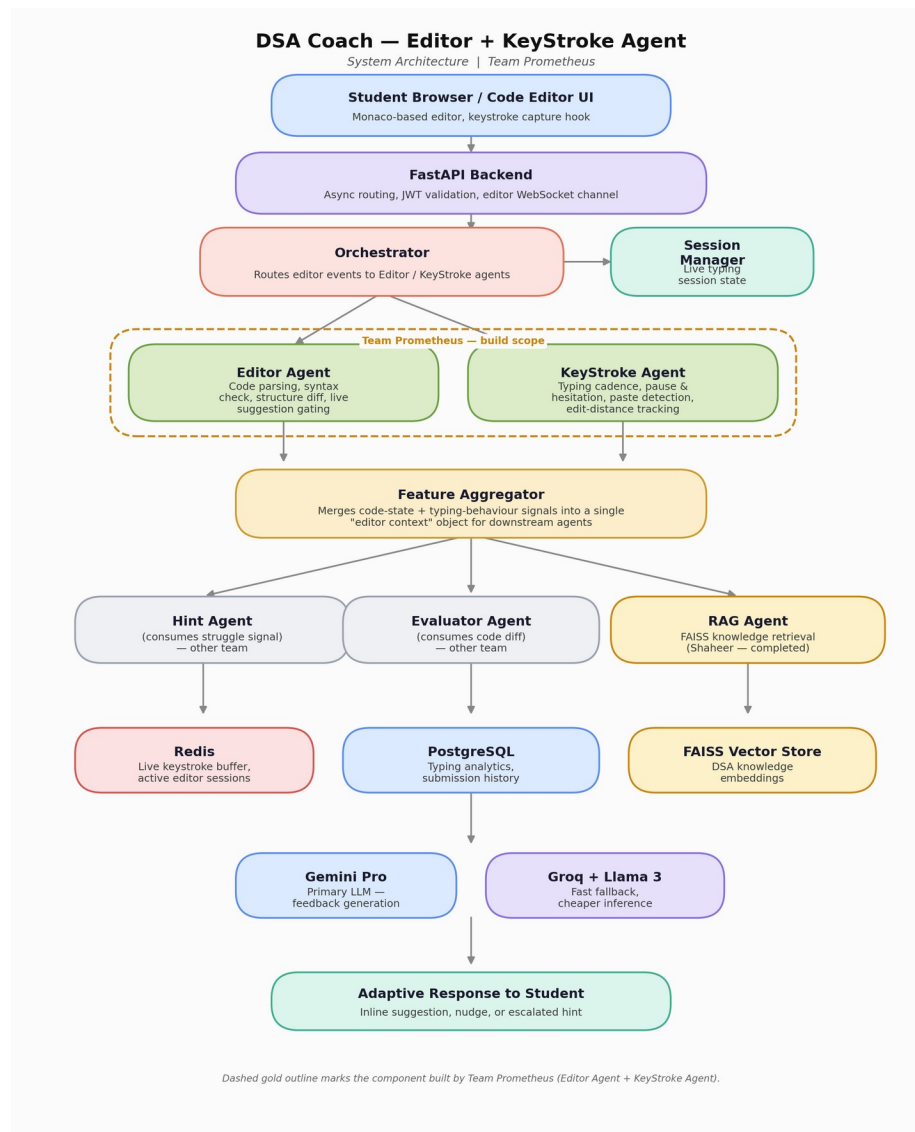


Figure 1 — DSA Coach system architecture, with the Editor + KeyStroke Agent (Team Prometheus build scope) outlined in gold.

6. Frontend / Code Editor UI

The frontend hosts a Monaco-based code editor instrumented with a lightweight capture hook that streams events to the backend without adding perceptible latency to typing.

Main functionalities

- Render the DSA problem statement alongside a live code editor.
- Stream keystroke and edit events over a WebSocket channel.
- Detect and tag paste / bulk-insert operations at the source.
- Display inline suggestions or nudges returned by downstream agents.
- Preserve a responsive, uninterrupted typing experience for the student.

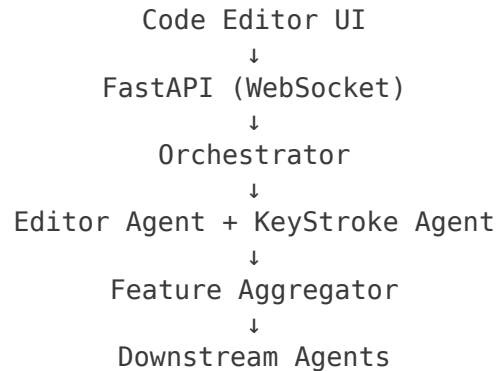
7. FastAPI Backend

FastAPI is the entry point for all editor traffic for this component.

Responsibilities

- WebSocket connection handling for live editor sessions
- JWT validation and rate limiting
- Request validation for editor and keystroke event payloads
- Routing events to the Orchestrator
- Forwarding aggregated features to downstream agents

Request Flow



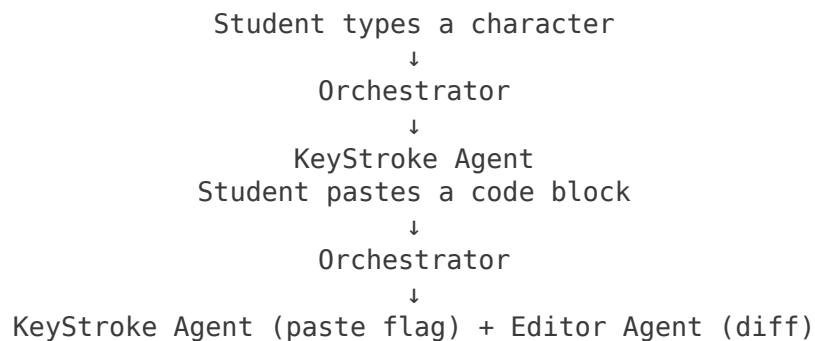
8. Orchestrator

The Orchestrator decides which agent processes an incoming editor event.

Routing logic

- Code-change event → Editor Agent
- Raw keystroke / timing event → KeyStroke Agent
- Paste event → KeyStroke Agent (flagged) and Editor Agent (structure check)
- Idle timeout → KeyStroke Agent (hesitation scoring)

Example



9. Editor Agent

The Editor Agent understands what the code is doing, structurally — independent of how it was typed.

Responsibilities

- Parse the current editor buffer into an AST-level or line-level representation

- Compute structural diffs between successive edits
- Detect syntax errors and incomplete constructs as they occur
- Gate which inline suggestions are safe to show without giving away the solution
- Emit a normalized "code state" object on every meaningful change

Output

- Current code state and diff summary
- Syntax validity
- Structural change classification (edit, refactor, rewrite, paste-insert)

10. KeyStroke Agent

The KeyStroke Agent understands how the student is typing — the behavioural signal layered underneath the code itself.

Responsibilities

- Measure typing cadence (keystrokes per minute, inter-key latency)
- Detect pauses and classify them as thinking, reading, or stuck
- Detect paste / bulk-insert events versus organic typing
- Track backspace and correction ratio
- Compute a rolling hesitation / struggle score for the session

Keystroke Feature Set

Feature	What it captures	Used for
Inter-key latency	Time between consecutive keystrokes	Typing cadence, confidence
Pause duration	Idle gaps above a threshold	Hesitation / struggle detection
Backspace ratio	Deletions relative to total keystrokes	Uncertainty, correction behaviour
Paste flag	Bulk-insert vs incremental typing	Authorship / integrity signal
Edit distance	Change size between snapshots	Rewrite vs incremental edit
Session WPM	Rolling words-per-minute of code entry	Engagement trend over time

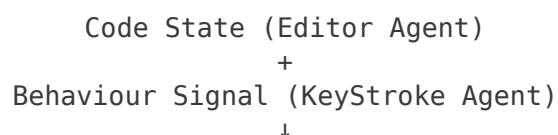
Output

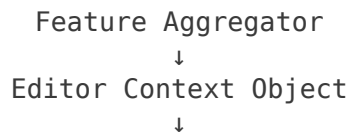
- Per-session struggle score (low / medium / high)
- Paste / authorship flag with confidence
- Typing cadence trend for the current problem

11. Feature Aggregator

The Feature Aggregator merges the Editor Agent's code state with the KeyStroke Agent's behavioural signal into a single editor-context object, refreshed continuously as the student works.

Aggregation Pipeline



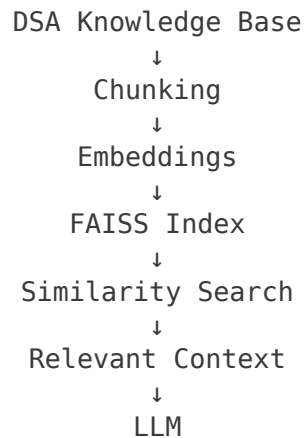


This aggregated object is what allows the Hint Agent (built by another GENAICH team) to offer help at the right moment — for example, only after sustained hesitation rather than on every pause — instead of reacting to code content alone.

12. RAG Integration (FAISS)

Retrieval-Augmented Generation grounds any behaviour-triggered nudge in actual DSA knowledge rather than a generic prompt. The RAG pipeline for this component — chunking, embedding, and FAISS indexing of DSA reference material — has been completed.

RAG Pipeline



Why FAISS

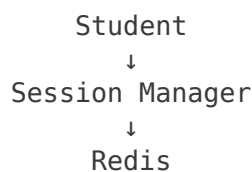
- Fast approximate nearest-neighbour search over DSA knowledge embeddings
- Lightweight, self-hosted vector index suited to the editor's low-latency requirements
- Already integrated and validated for this component

13. Session Manager and Redis

The Session Manager keeps the live editor session synchronized across the Editor Agent, KeyStroke Agent, and Feature Aggregator.

It manages

- Session ID and active problem context
- Rolling keystroke buffer for the current session
- Current struggle score and paste flags
- State handed off to downstream agents



14. PostgreSQL Database

PostgreSQL is the persistent store for data that must outlive a single session.

It stores

- Aggregated typing analytics per student and per problem
- Submission and code-state history
- Paste / authorship flags for audit purposes
- Longitudinal struggle trends used for progress tracking

Redis vs PostgreSQL

Redis	PostgreSQL
In-memory, fast	Persistent, relational
Active session buffer	Long-term analytics store
Frequently changing data	Historical / reportable data

15. LLM Layer

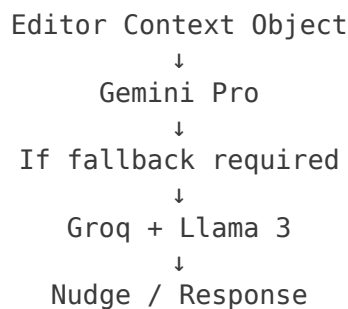
When the aggregated editor context indicates a nudge is warranted, the LLM layer turns it into natural language, grounded by the RAG-retrieved context.

Primary LLM — Gemini Pro

- High-quality, context-aware nudges
- Reasoning over combined code-state and behaviour signals

Fallback LLM — Groq + Llama 3

- Used when Gemini Pro is unavailable or a faster response is required



16. Privacy and Data Handling

Because this component observes typing behaviour directly, privacy is treated as a first-class design constraint rather than an afterthought.

Principles

- Store derived behavioural features (cadence, pause, paste flags) — not raw keystroke content or timing logs beyond the active session.

- Code snapshots are retained for evaluation purposes only, on the same terms as any DSA Coach submission.
- Struggle scores are used to calibrate help, never to penalize a student.
- All editor telemetry is scoped to the authenticated session and is not shared outside the DSA Coach platform.

17. End-to-End Example — Two Sum

Suppose a student opens the Two Sum problem and starts typing a solution.

Step 1 — Live Typing

Student types incrementally
↓
KeyStroke Agent: normal cadence, no pauses

Step 2 — Hesitation

Student pauses for 40 seconds mid-loop
↓
KeyStroke Agent: pause classified as "stuck"
↓
Struggle score rises to MEDIUM

Step 3 — Aggregation and Grounding

Feature Aggregator merges code state + struggle score
↓
RAG Agent retrieves Two Sum hint context from FAISS
↓
Editor Context Object passed to Hint Agent

Step 4 — Paste Detection

Student pastes a 15-line block
↓
KeyStroke Agent: paste flag = TRUE (high confidence)
↓
Editor Agent: structural diff = full-function replace
↓
Flag surfaced to Evaluator Agent for review

This creates a continuous behavioural loop underneath the usual code-submission loop:

Type → Observe → Score → Aggregate → Ground (RAG) → Nudge → Continue

18. Security Architecture

Security Features

9. WebSocket Authentication — every editor session is validated via JWT before events are accepted.
10. Rate Limiting — prevents flooding the event stream.
11. Session Isolation — each student's keystroke buffer is isolated in Redis.
12. Minimal Retention — raw keystroke timing is discarded once features are derived.
13. Paste-Flag Auditing — authorship flags are logged for downstream review, not automatic penalty.

14. Environment Variables — API keys and index paths are stored securely, never hard-coded.

19. Key Features

Behavioural Sensing

- Real-time keystroke dynamics capture
- Pause and hesitation classification
- Paste / bulk-insert detection

Code Understanding

- Live structural diffing
- Syntax validity checks
- Suggestion gating

Integration

- Feature aggregation for Hint / Evaluator agents
- RAG-grounded nudges (FAISS)
- Multi-LLM support (Gemini Pro, Groq + Llama 3)

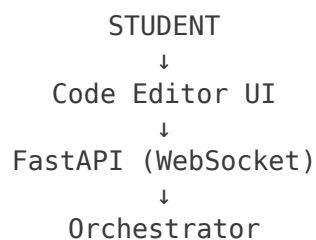
Data & Privacy

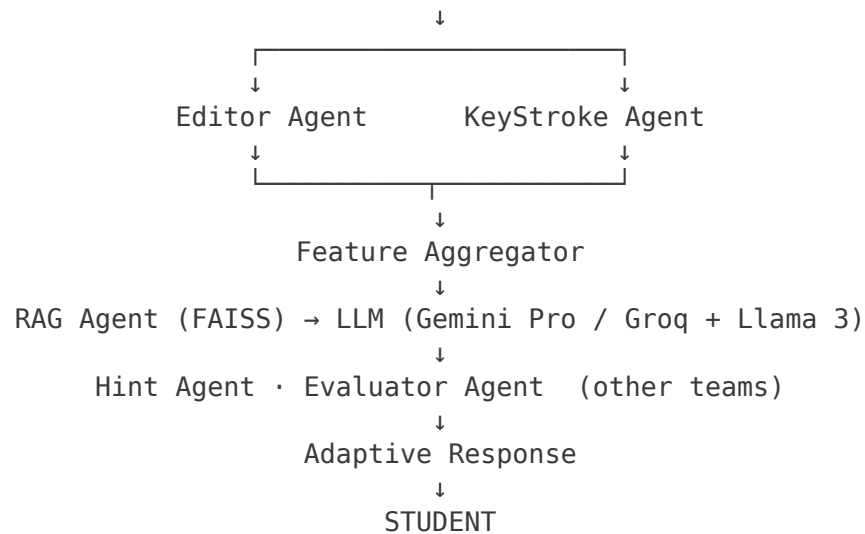
- Redis session buffering
- PostgreSQL analytics storage
- Feature-only retention policy

20. Complete Technology Stack

Layer	Technology
Frontend	Monaco-based Code Editor (Web)
Backend	FastAPI (WebSocket)
Orchestration	Custom event-routing Orchestrator
Behavioural Agent	KeyStroke Agent
Code Agent	Editor Agent
RAG	Retrieval-Augmented Generation
Vector Database	FAISS
Session Storage	Redis
Persistent Database	PostgreSQL
LLM	Gemini Pro
LLM Fallback	Groq + Llama 3
Authentication	JWT

21. Overall System Flow





Session Data → Redis Persistent Data → PostgreSQL

22. Advantages

- Reacts to how a student is working, not just what they eventually submit.
- Surfaces struggle early enough for a hint to be genuinely useful.
- Distinguishes authored code from pasted code with a clear, auditable signal.
- Grounds every nudge in real DSA knowledge via RAG, not a generic prompt.
- Keeps raw keystroke data out of long-term storage by design.
- Plugs directly into the existing DSA Coach orchestrator and downstream agents.

23. Conclusion

The Editor Agent and KeyStroke Agent give DSA Coach a behavioural layer that the rest of the platform can build on: a continuous, privacy-conscious read on how a student is actually working through a problem, aggregated and grounded through FAISS-based RAG, and handed off to the Hint and Evaluator agents in real time.

Team Prometheus combines FastAPI, a Monaco-based editor, FAISS, Redis, PostgreSQL, and multi-LLM support to move DSA Coach from evaluating final answers to understanding the process behind them:

Type → Observe → Score → Aggregate → Ground → Nudge → Improve

This makes the Editor + KeyStroke Agent the sensing foundation of an adaptive DSA mentor, rather than a passive code box.

Team Prometheus (GENAICH-003)

Members: Shaheer Khan, Piyush Mohapatra, Nirmalya Kumar Mohanty

Domain: DSA · Status: RAG (FAISS) completed